

Universidad EAFIT
Escuela de Ciencias Aplicadas e Ingeniería
Estructuras de Datos y Algoritmos

PRÁCTICA 1 — ESTRUCTURAS UNIDIMENSIONALES

Pilas y Colas en Sistemas Reales: Undo/Redo de un Editor de Código y Control de Tráfico en un Firewall — Implementación desde Cero y Análisis Formal de Complejidad

Estudiantes
Sara Castaño Suárez
Simón López Mesa
Antuan Alejandro Navarro Ortiz

Docente
Carlos Alberto Álvarez Henao

Materia
Estructura de Datos y Algoritmos

Lunes, 1 de Septiembre 2026
Medellín, Colombia

2. Introducción

Las estructuras de datos fueron creadas para organizar, almacenar y manipular datos en la memoria del computador de forma que se pueda acceder y/o modificar de forma eficaz. Hoy en día en la computación moderna se usan estructuras de datos unidimensionales que ayudan en la solución de problemas, en este trabajo veremos dos: El funcionamiento de una pila (LIFO) y el de una cola (FIFO). El primero mediante un editor de código que permita hacer operaciones de UNDO (deshacer) y REDO (rehacer), y el segundo ocurre en la frontera de una red: búfer de recepción y limitador de tasa de un firewall, que determinan si los paquetes serán o no aceptados.

En esta práctica se pretende desarrollar e implementar los dos problemas, verificar que el programa funcione correctamente, analizar la complejidad y determinar O , Ω y Θ , medir y mostrar los comportamientos que se tiene con diferentes tamaños en cada caso, comparar y concluir la relación que hay en la teoría y los resultados finales.

3. Contexto de los dos problemas

3.1 Problema 1: Sistema de Undo/Redo para un editor de código (Pila)

Se simula el funcionamiento del historial de cambios de un editor de código, en el cual un usuario o un asistente de IA puede realizar diferentes modificaciones sobre un documento, como insertar, eliminar o reemplazar contenido. El sistema debe registrar estas operaciones y permitir deshacer la última modificación realizada o rehacer una modificación previamente deshecha. Además, cuando se realiza una nueva edición después de uno o varios UNDO, el sistema debe eliminar el historial de REDO, siguiendo el comportamiento habitual de este tipo de editores.

3.2 Problema 2: Búfer de recepción y limitador de tasa de un firewall (Cola)

Se simula el funcionamiento de un firewall que recibe continuamente paquetes de datos, donde cada paquete contiene una marca de tiempo de llegada y un tamaño en bytes. El sistema debe almacenar temporalmente los paquetes en un búfer de capacidad limitada y rechazarlos cuando este se encuentre lleno. Además, debe controlar la cantidad de paquetes recibidos durante una ventana de tiempo determinada, rechazando aquellos que superen el límite establecido para evitar un exceso de tráfico.

4. Fundamentación teórica

4.1 Problema 1: Sistema de Undo/Redo para un editor de código (Pila)

La pila (Stack) es una estructura de datos abstracta que opera bajo el principio LIFO (Last In, First Out). Esto significa que el último elemento en ser insertado es el primero en ser retirado. Esta restricción de acceso es fundamental para garantizar que las operaciones se procesan en el orden inverso a su llegada, lo cual es el comportamiento central que requiere el sistema de Undo/Redo desarrollado en esta práctica.

Para mantener la integridad del modelo LIFO, la interacción con la estructura se restringe a tres operaciones elementales:

- `push(elemento)`: Inserta un nuevo elemento en la cima de la pila.
- `pop()`: Remueve el elemento que se encuentra en la cima. En nuestra implementación, arroja una excepción (`std::underflow_error`) si se intenta ejecutar sobre una pila vacía, garantizando la seguridad del programa.
- `top()` / `peek()`: Devuelve el valor del elemento en la cima sin removerlo, permitiendo consultar el estado actual antes de tomar una decisión.

Las operaciones descritas tienen una complejidad temporal constante en el caso global, esto debido a que siempre van a actuar sobre el mismo extremo de la estructura sin tener que recorrerla en su totalidad. Es menester aclarar que el análisis de la complejidad temporal en detalle se encuentra en la sección 7 del presente informe.

Adicionalmente, la pila puede implementarse de dos formas diferentes: como un arreglo dinámico o como una lista enlazada, la única diferencia entre estas dos implementaciones es en cómo ambas manejan la pila llena. La primera por un lado, redimensiona el arreglo al alcanzar la capacidad máxima. La segunda por otro lado, no tiene una condición puntualmente que le diga que la pila está llena tal como un arreglo dinámico. Lo que pasa es que para la lista enlazada eso jamás es un problema, nunca se va a topar con "no hay más espacio en el arreglo", porque no hay arreglo, cada elemento vive en su propio pedazo de memoria independiente.

Más allá del sistema de UNDO y REDO desarrollado en esta práctica, se puede encontrar este principio LIFO en otros contextos que hacen parte de la computación tales como: el historial de navegación en los navegadores web y la evaluación y conversión de expresiones matemáticas. Esto por mencionar dos de la amplia variedad de implementación.

4.2 Problema 2: Buffer de recepción y limitador de tasa de un firewall (Cola)

La cola (Queue) es una estructura de datos abstracta que opera bajo el principio FIFO (First In, First Out): el primer elemento en ser insertado es el primero en ser retirado. Esta disciplina de acceso refleja directamente el comportamiento que requiere el búfer de recepción del firewall, donde los paquetes deben procesarse en el mismo orden en que llegaron, y el mecanismo de ventana deslizante, donde las marcas de tiempo más antiguas se eliminan primero conforme dejan de ser relevantes.

La interacción con la estructura se restringe a las siguientes operaciones elementales:

- enqueue(elemento): Inserta un nuevo elemento al final de la cola.
- dequeue(): Remueve el elemento que se encuentra al frente de la cola.
- front() / rear(): Devuelven el valor del elemento al frente o al final de la cola, respectivamente, sin removerlo.

En una implementación circular sobre arreglo, los índices de frente (head) y final (tail) pueden coincidir tanto cuando la cola está completamente vacía como cuando está completamente llena, lo que introduce una ambigüedad si solo se dependiera de esos dos índices. Para evitar esta ambigüedad, la implementación mantiene un contador explícito del número de elementos actualmente almacenados (cantidad), de forma que "vacía" se determina cuando $\text{cantidad} == 0$ y "llena" cuando $\text{cantidad} == \text{capacidad}$, sin depender de la posición relativa de head y tail.

Las operaciones enqueue, dequeue, front y rear tienen complejidad temporal constante en el caso general, al operar siempre sobre los extremos de la estructura sin necesidad de recorrerla; el análisis formal de cada una, incluyendo el costo amortizado de la purga de marcas expiradas, se desarrolla en la Sección 7.

Adicional, el principio FIFO de la cola se usa en otros contextos de la computación como: la planificación de procesos en un sistema operativo, las colas de impresión, atención al cliente y callcenters, los búferes de entrada/salida en sistemas de red, y el manejo de mensajes en sistemas de mensajería asíncrona (colas de trabajo).

4.3 Listas enlazadas

Una lista enlazada es una estructura de datos lineal que consta de una secuencia de nodos. A diferencia de los arreglos, las listas enlazadas no requieren asignación de memoria contigua. En cambio, a cada nodo se le asigna dinámicamente su propio espacio de memoria. Los nodos se conectan mediante referencias, formando la estructura enlazada. Una ventaja de las listas enlazadas es que la inserción y eliminación de elementos al principio de la lista se puede realizar en tiempo constante, denotado como $O(1)$.

Esta eficiencia se debe a la capacidad de manipular directamente las referencias sin necesidad de desplazar los elementos como ocurre en los arreglos. Los tipos de datos en una lista enlazada pueden ser cualquiera de los tipos de datos disponibles que admiten un lenguaje de programación.

En la práctica, esta representación se usa en StackList (Problema 1) y ColaTimestampsLista (Problema 2), como segunda representación de cada TAD frente a su versión sobre arreglo.

Operaciones fundamentales:

- **Inserción:** crear un nuevo nodo y enlazarlo a la lista. Si se inserta en la cabeza (como en StackList), el costo es $O(1)$; si se inserta en una posición arbitraria, requiere recorrer la lista hasta esa posición.
- **Eliminación:** desenlazar un nodo, ajustando el puntero del nodo anterior para que apunte al siguiente del eliminado. Igual que la inserción, es $O(1)$ en la cabeza y $O(n)$ en una posición arbitraria.
- **Recorrido:** visitar cada nodo siguiendo los enlaces desde la cabeza, necesario para operaciones como búsqueda o impresión completa de la lista. Es $O(n)$.
- **Búsqueda:** recorrer la lista comparando cada nodo hasta encontrar el valor buscado (o llegar al final). $O(n)$ en el peor caso.

Frente a la representación en arreglo, la diferencia central está en el tipo de acceso: el arreglo permite acceso indexado directo $O(1)$ a cualquier posición, mientras que la lista enlazada solo permite acceso contiguo, por lo que alcanzar una posición arbitraria es $O(n)$. En los extremos, sin embargo, la lista enlazada iguala o supera al arreglo: insertar o eliminar en la cabeza es $O(1)$ sin desplazar elementos, mientras que en un arreglo esa misma operación al inicio requiere desplazar todos los demás. A cambio, cada nodo de la lista enlazada añade overhead de memoria por el puntero de referencia, ausente en el arreglo.

Es importante decir que al igual que la pila, la cola puede implementarse ya sea sobre un arreglo circular o como lista enlazada. En el transcurso del informe se verán los costos y demás detalles.

5. Diseño de las soluciones (TAD e implementaciones)

5.1 Problema 1: Sistema de Undo/Redo para un editor de código (Pila)

TAD Pila

Operación	Lo que hace
push(x)	Inserta el elemento x en el final de la pila
pop()	Elimina el elemento que está al final de la pila
top() / peek()	Consulta el elemento que está en el final de la pila
isEmpty()	Dice si la pila tiene o no elementos

Estructuras Implementadas:

- StackArray (src/stack_array.hpp/cpp): Pila genérica implementada sobre un arreglo dinámico. Incluye lógica de redimensionamiento automático (resize()) cuando se alcanza la capacidad máxima.
- StackList (src/stack_list.hpp/cpp): Pila genérica sobre una lista simplemente enlazada con nodos gestionados dinámicamente en el heap. Cumple el requisito de doble representación estructural.
- UndoRedoManager (src/undoredo.hpp/cpp): Gestor del historial de operaciones de texto utilizando el patrón de dos pilas (undoStack y redoStack). Aplica y revierte acciones como inserciones y eliminaciones leyendo un flujo de eventos.

Justificación duplicación de capacidad: Cuando `currentSize == capacity` al momento de un push, se redimensiona la capacidad duplicando `capacity*2`. La duplicación del arreglo hace que el costo de las operaciones push sea $O(n)$ lo que se traduce en un costo amortizado de $O(1)$ por cada operación.

El proyecto requirió implementar la pila bajo dos arquitecturas distintas para analizar sus ventajas de rendimiento y gestión de memoria.

Pila sobre Arreglo Dinámico (Stack Array)

Utiliza un bloque de memoria contigua.

- Gestión de Capacidad y Redimensionamiento: A diferencia de un arreglo estático que lanza un error de "pila llena" (Stack Overflow) al alcanzar su límite, esta implementación utiliza un mecanismo de

redimensionamiento dinámico. Cuando el índice actual (currentSize) iguala la capacidad, el sistema reserva un nuevo bloque de memoria con el doble de tamaño, copia los elementos existentes y libera el bloque anterior.

- Ventaja: Excelente localidad de referencia en caché, lo que hace que las lecturas secuenciales sean extremadamente rápidas a nivel de hardware.

Pila sobre Lista Enlazada (StackList)

Utiliza nodos dispersos en el heap (memoria dinámica), donde cada nodo contiene el valor y un puntero al nodo anterior/siguiente.

- Condición de Llenado: No sufre de "pila llena" ni requiere redimensionamiento, ya que crece dinámicamente nodo por nodo hasta que se agota la memoria RAM del sistema.
- Desventaja: Mayor consumo de memoria debido al puntero extra en cada nodo y peor rendimiento en caché por la fragmentación de la memoria.

Esta misma interfaz puede satisfacer exitosamente las dos representaciones de arreglo dinámico y lista enlazada. Desde la perspectiva de UndoRedoManager (undoredo.cpp), ambas representaciones exponen la misma interfaz y el resultado secuencial del uso de EDIT, UNDO, REDO es igual. Esto se puede verificar directamente en la sección 8.1 obteniendo el mismo resultado en los 14 casos de prueba (7 por StackArray y 7 por StackList).

5.2 Problema 2: Buffer de recepción y limitador de tasa de un firewall (Cola)

TAD Cola

Operación	Lo que hace
enqueue(x)	Insertar el elemento x al final de la cola
dequeue()	Elimina el primer elemento de la cola
front() / rear()	Consulta el frente o lo último de la cola
isFull() / is Empty()	Indica si está llena o vacía la cola

Estructuras Implementadas:

- BuferPaquetes (src/queue_circular.hpp/.cpp): cola circular sobre arreglo, capacidad fija C. Búfer de recepción de paquetes.
- ColaTimestamps (src/queue_timestamps.hpp/.cpp): cola circular sobre arreglo, guarda timestamps para la ventana deslizante del limitador de tasa.
- ColaTimestampsLista (src/queue_list.hpp/.cpp): misma interfaz que ColaTimestamps, implementada con lista enlazada. Cumple el requisito de segunda representación (sección 7 del enunciado).
- ratelimiter (src/ratelimiter.hpp/.cpp): conecta las colas y decide, por cada paquete, si se ACEPTA, se rechaza por BÚFER LLENO, o se rechaza por LÍMITE DE TASA.

Decisión de diseño — borde de la ventana $t = t_0 + T$

Se definió que el borde queda incluido dentro de la ventana (no se purga en ese instante exacto). Se implementa en purgarExpirados con la condición estricta valorFrente < límite (no <=). Ver Caso de prueba 5 para la evidencia.

Arreglo circular de capacidad fija (ColaTimestamps): un bloque de memoria contiguo de tamaño C gestionado manualmente, con índices head y tail que avanzan módulo C y un contador cantidad que distingue el caso "cola llena" del caso "cola vacía" (ambos comparten head == tail). Añade la operación isFull(), propia de una representación acotada.

Lista enlazada simple (ColaTimestampsLista): una cadena de nodos con punteros head y tail; enqueue crea un nodo e inserta por la cola, dequeue desengancha y libera el nodo de la cabeza. No tiene capacidad predefinida ni operación isFull().

Qué permanece igual desde la perspectiva de quien usa la estructura. El módulo `ratelimiter`, que consume la cola para el control por ventana deslizante, invoca exactamente el mismo conjunto de operaciones (`enqueue`, `dequeue`, `front`, `isEmpty`, `size`, `purgarExpirados`) en ambos casos. El comportamiento observable es idéntico: dado el mismo flujo de paquetes y los mismos parámetros T y L , la purga deja la cola en el mismo estado y la decisión de aceptar o rechazar cada paquete es la misma, cualquiera sea la representación. El código cliente no necesita conocer si por dentro hay un arreglo o una lista.

Qué cambia. Únicamente la representación interna y sus costes asociados:

Aspecto	Arreglo circular	Lista enlazada
Capacidad	fija, definida al construir	ilimitada (acotada solo por la memoria disponible)
enqueue con estructura llena	devuelve fallo (false) sin insertar	no aplica: siempre inserta
Memoria por elemento	un long	un long más un puntero (overhead de enlace)
Localidad de acceso	alta (bloque contiguo, favorable a la caché)	baja (nodos dispersos en el heap)
Reserva/liberación	una sola vez, al construir y destruir	una asignación y una liberación por elemento

6. Pseudocódigo de las operaciones fundamentales

Nota: Los pseudocódigos se encuentran en el repositorio. Pseudocódigo `queue &`
Pseudocódigo `pila.txt`

7. Análisis de complejidad

Nota aclaratoria para entender la diferencia entre $O(1)$ "normal" y $O(1)$ amortizado: $O(1)$ garantiza que cada operación individual tarda un tiempo constante, mientras que $O(1)$ amortizado significa que el promedio de tiempo por operación es constante a lo largo de una secuencia de operaciones, aunque alguna ejecución individual pueda ser mucho más lenta.

7.1 Tabla de complejidad - problema 1 (Pila)

Operación	Mejor caso	Caso promedio	Peor caso	Complejidad espacial	Justificación exigida
push (arreglo, sin redimensionar)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Costo fijo de escritura e incremento de índice // Lo que pasa es que <code>currentSize</code> es menor que la capacidad <code>capacity</code> , por lo que no se logra cumplir el condicional <code>if</code> . Esto hace que independientemente de la cantidad de elementos que ya hay en la memoria tome el mismo tiempo, por que el arreglo está en memoria

					contigua.
push (arreglo, con redimensionamiento)	$O(1)$	$O(1)$ Amortizado	$O(n)$	$O(n)$	Distinguir costo puntual del evento de copia vs. costo amortizado sobre n operaciones // Aquí el if si se cumple <code>if (currentSize == capacity)</code> . El ciclo va a recorrer los elementos que hay y los copiará en un arreglo nuevo. entonces si tengo n elementos, tengo que copiar esos n , por lo que el costo de esta operación puntual es de $O(n)$. Ojo que esto solo pasa cuando el arreglo está lleno y no siempre en cada push. y como la política es duplicar la capacidad (no sumar una cantidad fija), después de cada <code>resize</code> hacen falta el doble de push que la vez anterior para que se vuelva a llenar y que pase otro <code>resize</code> . Esto significa que, se distribuye el costo de esa copia de n elementos entre los n push que pasaron desde el <code>resize</code> pasado. El costo adicional por cada push es de $O(1)$. Por eso, independientemente de que el peor caso de el push que dispara el <code>resize</code> es $O(n)$, el costo amortizado sobre una larga secuencia de estas operaciones es de $O(1)$ por cada push que pasa.
push (lista enlazada)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Costo de creación de nodo e inserción en cabeza // Nunca habrá que recorrer toda la lista ni copiar nada existente, por eso se coloca $O(1)$ en todos los casos. Esto por que solo habría que cambiar 2 referencias: conectar el nuevo nodo al primero y actualizar la cabeza. No requiere desplazar ningún dato de la memoria.
pop / peek (ambas representaciones)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Acceso/eliminación en el extremo activo // <code>pop()</code> le resta al tamaño actual en un paso al igual que <code>peek</code> funciona como un acceso directo. No se tiene que recorrer por completo, por eso siempre será $O(1)$.
isEmpty	$O(1)$	$O(1)$	$O(1)$	$O(1)$	// Es solo una única comparación: está o no lleno? <code>currentSize == 0?</code>

7.2 Tabla de complejidad - problema 2 (Cola)

Operación	Mejor caso	Caso promedio	Peor caso	Complejidad espacial	Justificación exigida
enqueue (cola circular)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Costo fijo si $\text{tail} \neq \text{head} \pmod{C}$ tras la operación // No importa cuántos elementos hay, siempre van a ocurrir estas 3 cosas: $\text{datos}[\text{tail}] = p$; $\text{tail} = (\text{tail} + 1) \% \text{capacidad}$; $\text{cantidad}++$; y no tienen que recorrer nada. Aquí como la capacidad es fija, en caso de estar llena se rechaza.
dequeue (cola circular) miento)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Costo fijo de avance de índice head // Aquí pasa lo mismo con enqueue solo que decrece --
Purga de marcas expiradas (por paquete)	$O(1)$	$O(1)$ amortizado	$O(k)$ $k = \text{constante}$	$O(1)$	Distinguir costo puntual (n° de elementos expirados en esa llamada) de costo amortizado sobre el total de n paquetes procesados // Lo que pasa con el peor caso es que si hay k timestamps vencidos ya acumulados hay un coste de $O(k)$, por otro lado, el amortizado $O(1)$ implica que cada timestamp solo pueda purgarse una única vez. El total de los deque no va a superar el valor de n , por ende sería $O(1)$ llamada en promedio
isFull / isEmpty	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Regla para distinguir cola llena de cola vacía en representación circular // Solo se requiere de una única comparación de cantidad. $\text{cantidad} == \text{capacidad}$ y/o $\text{cantidad} == 0$.

Explicación / Justificación: En la casilla de justificación exigida está yuxtapuesto a partir de //.

7.3 Análisis global

Desde la tabla del problema 1 se puede decir que procesar un archivo de n eventos implica ejecutar n operaciones $O(1)$ o $O(1)$ amortizado sobre las pilas (Tabla 7.1), dando una complejidad temporal total de $O(n)$. En el peor caso, si el archivo consiste únicamente en EDIT sin ningún UNDO, undoStack llega a almacenar los n eventos, dando una complejidad espacial total de $O(n)$. El análisis requiere argumento amortizado por el redimensionamiento de StackArray : aunque el push que dispara un resize cuesta $O(n)$ en ese instante puntual,

la duplicación de capacidad hace que ese costo se distribuya entre cada vez más operaciones, dando un costo promedio de $O(1)$ por push (ver justificación completa en la fila "push con redimensionamiento", Tabla 7.1). La aclaración se puede verificar desde la parte experimental, Sección 10.1.

Por otro lado, la tabla de complejidad del problema 2 se deduce que procesar un archivo de n paquetes implica n operaciones $O(1)$ o $O(1)$ amortizado sobre las colas (Tabla 7.2), dando una complejidad temporal total de $O(n)$. BuferPaquetes tiene complejidad espacial $O(1)$ respecto a n (capacidad fija C); ColaTimestamps puede crecer hasta $O(n)$ en el peor caso, si ningún timestamp llega a purgarse durante la ejecución. Una llamada a purgarExpirados puede costar $O(k)$ en el peor caso puntual (k timestamps vencidos para ese momento), pero como cada timestamp solo se purga una vez en toda su vida útil, el costo total de todas las purgas no supera n a lo largo de la ejecución, dando un costo promedio de $O(1)$ por paquete (ver justificación completa en la fila "Purga de marcas expiradas", Tabla 7.2). La aclaración se puede verificar desde la parte experimental, Sección 10.2.

8. Casos de prueba

8.1 Problema 1: Sistema de Undo/Redo para un editor de código (Pila)

StackArray

Caso	Archivo de entrada	Resultado esperado	Resultado obtenido
1 Secuencia normal mixta	tests/p1_caso1_secuencia_normal.txt	Documento final consistente con la secuencia; UNDO/REDO exitosos según corresponda	(a) Estado final del documento: "DRAGPAPAYAON" (c) Elementos finales en Pila Undo: 2 (c) Elementos finales en Pila Redo: 0
2 UNDO sin ediciones previas	tests/p1_caso2_undo_sin_ediciones.txt	UNDO reportado como no-op (pila Undo vacía); no debe abortar el programa	(a) Estado final del documento: "" (c) Elementos finales en Pila Undo: 0 (c) Elementos finales en Pila Redo: 0
3 Una edición, UNDO, REDO	tests/p1_caso3_edit_undo_redo.txt	Documento vuelve al estado post-EDIT tras el REDO	(a) Estado final del documento: "ESTERNOCLEIDOMASTOIDEO" (c) Elementos finales en Pila Undo: 1 (c) Elementos finales en Pila Redo: 0
4 Edición inmediata después de un UNDO (verificar vaciado de Redo)	tests/p1_caso4_edit_tras_undo.txt	Pila Redo queda en 0 elementos tras el nuevo EDIT	(a) Estado final del documento: "CHANCLRasputinAS" (c) Elementos finales en Pila Undo: 2 (c) Elementos finales en Pila Redo: 0
5 N ediciones seguidas de N deshacer consecutivos	tests/p1_caso5_n_edits_n_undos.txt	Documento vuelve al estado inicial; pila Undo en 0 elementos	(a) Estado final del documento: "" (c) Elementos finales en Pila Undo: 0 (c) Elementos finales en Pila Redo: 5
6 Más REDO que elementos disponibles	tests/p1_caso6_redo_excedente.txt	Los REDO válidos se ejecutan; los sobrantes se reportan como no-op	(a) Estado final del documento: "interestelar" (c) Elementos finales en Pila Undo: 1 (c) Elementos finales en Pila Redo: 0

7 Prueba dedicada al crecimiento de capacidad de la versión de arreglo	tests/p1_caso7_crecimiento_capacidad.txt	Ocurre al menos un resize; el documento y las pilas quedan correctos tras el resize	(a) Estado final del documento: "OnctGfrPojoAmperioshmniolsulosrolataubidioluorermaniouiuloobreitrogenoxigeno" (c) Elementos finales en Pila Undo: 12 (c) Elementos finales en Pila Redo: 0 Importante aclarar que se evidencia indirectamente un redimensionamiento(no se imprime nada en la terminal que muestre esto).
---	--	---	---

StackList

Caso	Archivo de entrada	Resultado esperado	Resultado obtenido
1 Secuencia normal mixta	tests/p1_caso1_secuencia_normal.txt	Documento final consistente con la secuencia; UNDO/REDO exitosos según corresponda	(a) Estado final del documento: "DRAGPAPAYAON" (c) Elementos finales en Pila Undo: 2 (c) Elementos finales en Pila Redo: 0
2 UNDO sin ediciones previas	tests/p1_caso2_undo_sin_ediciones.txt	UNDO reportado como no-op (pila Undo vacía); no debe abortar el programa	(a) Estado final del documento: "" (c) Elementos finales en Pila Undo: 0 (c) Elementos finales en Pila Redo: 0
3 Una edición, UNDO, REDO	tests/p1_caso3_edit_undo_redo.txt	Documento vuelve al estado post-EDIT tras el REDO	(a) Estado final del documento: "ESTERNOCLEIDOMAS TOIDEO" (c) Elementos finales en Pila Undo: 1 (c) Elementos finales en Pila Redo: 0
4 Edición inmediatamente después de un UNDO (verificar vaciado de Redo)	tests/p1_caso4_edit_tras_undo.txt	Pila Redo queda en 0 elementos tras el nuevo EDIT	(a) Estado final del documento: "CHANCLRasputinAS" (c) Elementos finales en Pila Undo: 2 (c) Elementos finales en Pila Redo: 0
5 N ediciones seguidas de N deshacer consecutivos	tests/p1_caso5_n_edits_n_undos.txt	Documento vuelve al estado inicial; pila Undo en 0 elementos	(a) Estado final del documento: "" (c) Elementos finales en Pila Undo: 0 (c) Elementos finales en Pila Redo: 5
6 Más REDO que elementos disponibles	tests/p1_caso6_redo_excedente.txt	Los REDO válidos se ejecutan; los sobrantes se reportan como no-op	(a) Estado final del documento: "interestelar" (c) Elementos finales en Pila Undo: 1 (c) Elementos finales en Pila Redo: 0

7 Prueba dedicada al crecimiento de capacidad de la versión de arreglo	tests/p1_caso7_crecimiento_capacidad.txt	Ocurre al menos un resize; el documento y las pilas quedan correctos tras el resize	(a) Estado final del documento: "OnctGfrPojoAmperiosh mniosulosrolataaubidiolu orermaniouliobreitrogen oxigeno" (c) Elementos finales en Pila Undo: 12 (c) Elementos finales en Pila Redo: 0 Nota: no se aplica redimensionamiento debido al comportamiento del StackList.
---	--	---	---

8.2 Problema 2: Búfer de recepción y limitador de tasa de un firewall (Cola)

#	Caso	Archivo de entrada	Parámetros (C, T, L)	Resultado esperado	Resultado obtenido
1	Flujo normal por debajo de C y L	tests/p2_caso1_flujo_normal.txt	C=100, T=10, L=5	Todos los paquetes aceptados	Aceptados: 5
2	Consulta sobre búfer vacío	(archivo vacío)	C=100, T=10, L=1	0 en todos los contadores	Procesados: 0, Aceptados: 0
3	Búfer exactamente lleno + paquete adicional	tests/p2_caso3_bufer_lleno.txt	C=2, T=..., L=...	El paquete extra se rechaza por búfer lleno	Rechazados por búfer: 1
4	Ráfaga que excede L dentro de T	tests/p2_caso4_rafaga_excede_L.txt	C=100, T=..., L=...	Los paquetes que exceden L se rechazan por tasa	Rechazados por tasa: 2
5	Paquete exactamente en el borde $t = t_0 + T$	tests/p2_caso5_borde_ventana.txt	C=100, T=10, L=1	El segundo paquete se rechaza por tasa (el borde cuenta dentro de la ventana)	Aceptados: 1, Rechazados por tasa: 1
6	Deque sobre búfer vacío	— (prueba directa, sin archivo)	—	dequeue() retorna false sin crashear	"dequeue() retorno false: el bufer estaba vacío, como se esperaba."
7	Un único paquete	tests/p2_caso7_un_paquete.txt	C=100, T=10, L=5	El paquete se acepta	Aceptados: 1

9.Experimentación

9.1 Problema 1: Sistema de Undo/Redo para un editor de código (Pila)

Carpeta /results -> REPORTE (Para más detalle)

Especificaciones de la máquina: CPU AMD RYZEN 7 260, 32GB RAM, WINDOWS 11

Configuración: g++ -std=c++17 -O2, **semilla del generador = 42**

Comando: .generador_eventos.exe

StackArray

n	Media (ms)	Desv. estándar	Elementos finales Undo	Elementos finales Redo
1000	13.06ms	0.72	562	0
10000	57.71ms	2.02	5654	1
100000	559.51ms	11.77	56775	3
1000000	5453.16ms	36.07	573381	0

StackList

n	Media (ms)	Desv. estándar	Elementos finales Undo	Elementos finales Redo
1000	7.35ms	0.93	562	0
10000	58.95ms	3.02	5654	1
100000	551.83ms	16.39	56775	3
1000000	5558,72ms	257.26	573381	0

Nota: las 5 iteraciones de cada caso se encuentren en /results – RESULTADOS P1 EXPERIMENTACIÓN. En estas tablas solo se ilustra el resultado tras operar esos datos.

Comparando ambas tablas de StackArray y StackList podemos retomar lo visto en la sección 5.1, donde ambas representaciones muestran la misma interfaz y generan los mismos resultados que se pueden ver en el resumen de las tablas o bien, volviendo a generarlos desde el programa.

Justificación de los tamaños: Los tamaños se tomaron desde los recomendados, estos siendo: $n \in \{1\ 000; 10\ 000; 100\ 000; 1\ 000\ 000\}$ eventos. Como cada operación individual de la pila son $O(1)$ o $O(1)$ amortizado, se requiere números de orden 10^5 a 10^6 para que el tiempo total supere el ruido que genera la misma medición del sistema debido al overhead del reloj y la variabilidad del sistema operativo utilizado

El generador de eventos para el problema 1 se encuentra en el /src y los documentos con los datos sintéticos en .txt se encontraran en /tests para poder acceder a ellos desde el menú una vez ejecutado el main..

9.2 Problema 2: Búfer de recepción y limitador de tasa de un firewall (Cola)

Carpeta /results -> REPORTE (Para más detalle)

Especificación de la máquina: AMD Athlon Gold 3150U, 12 GB RAM, Windows 10 IoT Enterprise LTSC 21H2 (build 19044.5131).

Configuración: g++ -std=c++17 -O2, **semilla del generador = 42**

Comando: .experimento.exe data/paquetes_<n>.txt 777777 66666 3333

Parámetros fijos: C = 777777, T = 66666 ms, L = 3333

n	Media (ms)	Desv. estándar	Aceptados	Rechazados por búfer	Rechazados por tasa
---	------------	----------------	-----------	----------------------	---------------------

1 000	1.45	0,35	1000	0	0
10 000	10.11	1,89	6666	0	3334
100 000	121.57	22.67	53328	0	46672
1 000 000	1425,74	374,96	525251	0	474749

Tómese esta tabla como guía de los resultados experimentales obtenidos, para mayor información, puede seguir: Carpeta /results -> REPORTE (Para más detalle).

Tiempo medio por paquete (Media / n):

n = 1000 -> 1.45 us/paq
n = 10000 -> 1.01 us/paq
n = 100000 -> 1.22 us/paq
n = 1000000 -> 1.43 us/paq

El tiempo total crece de forma aproximadamente lineal con n (factor x10 en tamaño -> factor ~7x-12x en tiempo), consistente con el costo Theta(n) amortizado deducido en la Sección 8. El costo por paquete se mantiene en el orden de 1-1.5 us en todas las escalas.

Nota: con C=777777 el bufer nunca se llena (el total de paquetes aceptados, 525251 en el caso mayor, queda muy por debajo de la capacidad y el enunciado no exige un consumidor que lo vacíe). Por eso "Rechazados por bufer" es 0 en todos los tamaños y la única causa de rechazo activa es el limitador de tasa por ventana deslizante (L=3333 paquetes en T=66666 ms). La fracción de aceptados se estabiliza cerca del 52-53% en los archivos grandes, cuando la ventana llega a saturación sostenida.

Justificación de los tamaños: Los tamaños se tomaron desde los recomendados, estos siendo: $n \in \{1\,000; 10\,000; 100\,000; 1\,000\,000\}$ eventos/paquetes. Como cada operación individual de la cola son $O(1)$ o $O(1)$ amortizado, se requiere números de orden 10^5 a 10^6 para que el tiempo total supere el ruido que genera la misma medición del sistema debido al overhead del reloj y la variabilidad del sistema operativo utilizado.

10. Comparación teórico-experimental

10.1 Problema 1: Sistema de Undo/Redo para un editor de código (Pila)

Crecimiento observado vs. esperado: El tiempo total crece de forma casi que lineal con n en ambas representaciones (factor x10 en tamaño produce un factor cercano a x10 en tiempo entre $n=10000$ y $n=1000000$, tanto en StackArray como en StackList), consistente con el costo $O(n)$ total visto en la Sección 7.3 (n operaciones $O(1)$ amortizado). El tiempo por evento se estabiliza alrededor de 5.5 microsegundos a partir de $n=10000$ en ambas representaciones (Sección 9.1).

Efecto de constantes ocultas por la notación asintótica: En $n=1000$, StackArray muestra un costo por evento notablemente más alto (13.06 microsegundos) que StackList (7.35 microsegundos), pese a que ambas son $O(1)$ amortizado según la teoría. Esto muestra que la notación asintótica oculta constantes: a escalas pequeñas, el overhead fijo de arranque del programa y la baja cantidad de eventos para cuadrar ese overhead pesan proporcionalmente más que la diferencia real de complejidad entre representaciones, la cual solo se vuelve despreciable a partir de $n=10000$.

Efecto de la jerarquía de memoria: A $n=1000000$, StackArray presenta una desviación estándar considerablemente menor (36.07 ms) que StackList (257.26 ms), pese a tener una media similar. Esto es consistente con lo planteado en la Sección 5.1: StackArray, al usar memoria contigua, tiene mejor localidad de referencia en caché y un comportamiento más predecible; StackList, con nodos dispersos en el heap, sufre más variabilidad por peor localidad de acceso y el costo variable de cada asignación dinámica de memoria.

Condiciones de ejecución: CPU AMD Ryzen 7 260, 32 GB RAM, Windows 11, compilado con g++ -std=c++17 -O2. – Computador de Sara.

Evidencia de comportamiento amortizado: Las pruebas miden el tiempo total de procesamiento por archivo, no el costo de cada operación individual, por lo que no se capturan directamente los picos puntuales de cada `resize` de `StackArray`. Sin embargo: aunque cada `resize` cuesta $O(n)$ en el momento en que ocurre, el crecimiento total se mantiene lineal a través de los cuatro tamaños medidos. Esto es consistente con el análisis de la Sección 7.1 (`push` con redimensionamiento, $O(1)$ amortizado).

10.2 Problema 2: Búfer de recepción y limitador de tasa de un firewall (Cola)

Crecimiento observado vs. esperado: El tiempo total crece de forma casi que lineal con n (factor $\times 10$ en tamaño produce un factor entre $\times 7$ y $\times 12$ en tiempo), consistente con el costo $O(n)$ amortizado total deducido en la Sección 7.3. El costo por paquete se mantiene en el orden de 1-1.5 microsegundos en todos los n medidos.

Efecto de constantes ocultas por la notación asintótica: El tiempo por paquete no decrece de forma uniforme con n (1.45 microsegundos en $n=1000$, 1.01 microsegundos en $n=10000$, subiendo de nuevo a 1.22-1.43 microsegundos en $n=100000$ y $n=1000000$) pese a que la operación es $O(1)$ amortizado en todos los casos. Esta variación refleja ruido de medición del sistema operativo y efectos de caché que la notación asintótica no captura, más que un cambio real en la complejidad del algoritmo.

Efecto de la jerarquía de memoria: A diferencia del Problema 1, la experimentación de tiempos del Problema 2 (Sección 9.2) se corrió únicamente sobre la representación de arreglo circular (`ColaTimestamps`); no se midió el tiempo de `ColaTimestampsLista`. Por lo tanto, la comparación de costos de memoria entre arreglo y lista enlazada para este problema se sustenta en el análisis teórico de la Sección 5.2 (localidad de acceso, overhead de puntero por nodo) y no en los números experimentales.

Condiciones de ejecución: AMD Athlon Gold 3150U, 12 GB RAM, Windows 10 IoT Enterprise LTSC 21H2, compilado con g++ -std=c++17 -O2. — Computador de Simón.

Nota: la experimentación del Problema 2 se ejecutó en un equipo distinto al del Problema 1, por restricciones del sistema operativo del equipo original que no se resolvieron a tiempo. La comparación válida es dentro de cada problema, no entre problemas.

Evidencia de comportamiento amortizado: Con $C=777777$ el búfer nunca se llena, por lo que la purga de marcas expiradas (la operación con costo puntual $O(k)$ analizada en la Sección 7.2) es la única fuente de trabajo variable por paquete. El crecimiento lineal observado en el tiempo total, sin saltos abruptos ni un crecimiento lineal, es consistente con que cada timestamp se purga una única vez en toda la ejecución (Sección 7.2), confirmando en la práctica el argumento de amortización usado en el análisis teórico.

11. Conclusiones

1. Durante la implementación se detectó un ****bug crítico de desbordamiento de memoria**** en `BuferePaquetes::enqueue()`: la línea incrementada por error la variable `capacidad` en lugar de `cantidad`, causando que `tail` termina apuntando fuera del arreglo real tras cientos de paquetes procesados (crash silencioso, sin excepción). Se diagnosticó aislando `BuferePaquetes` y `ColaTimestamps` por separado (comentando su uso uno a la vez) hasta confirmar la causa raíz, y se corrigió cambiando `capacidad++` por `cantidad++`. Verificado con archivos de hasta 1,000,000 de paquetes sin fallos posteriores.
2. La comparación experimental entre `StackArray` y `StackList` (Sección 9.1) confirmó lo planteado en la Sección 5.1: ambas producen resultados idénticos en los 14 casos de prueba y en las 4 escalas medidas. Sí existen diferencias de rendimiento entre ellas, pero son insignificantes frente al tiempo total y solo se hacen visibles al medir con precisión, no a simple vista. Esto evidencia que dos implementaciones con la misma medida asintótica pueden diferir en las constantes ocultas.
3. El Problema 2 permitió ver, sin las capas de complejidad de una red real, el mecanismo detrás de algo que ocurre cada vez que se visita cualquier página de internet: un firewall decidiendo, paquete por paquete, si algo se acepta, se descarta por saturación o se rechaza por exceso de tráfico, funcionando también como una primera línea de defensa frente a abusos como los ataques de denegación de servicio.

4. Esta práctica evidenció cómo estructuras aparentemente simples como lo son una pila y/o una cola son la base de sistemas mucho más grandes, desde el historial de un editor de texto hasta la seguridad de una red. El trabajo en equipo, con implementación, pruebas y documentación divididas, permitió además reforzar la teoría mediante la experimentación directa y el intercambio de hallazgos entre los integrantes.

12. Uso de herramientas de IA

El uso de IA para el informe fue en su mayoría preguntas que dicen: tengo esto, que le falta para estar completo y que cumpla con lo solicitado? Asimismo que se usó para aclarar conceptos, explicaciones e interpretaciones de código para lograr la mayor profundidad posible en el informe.

En el siguiente documento se puede encontrar resumen, conversaciones e interacciones de la IA de el equipo y de forma individual:

<https://docs.google.com/document/d/1hRs1o9ppDtLT9LRtwdT7ZQQXYKVSOt7TLDOxKtn8zXs/edit?usp=sharing>

13. Referencias

https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1262/lectures/05-stack-queue/?utm_source=chatgpt.com

<https://cdacamar.github.io/data%20structures/algorithms/benchmarking/text%20editors/c++/rethinking-undo/>

<https://docencia.eafranco.com/materiales/estructurasdedatos/02/Tema02.pdf>

<https://www.luisllamas.es/que-es-una-queue/>

<https://intellipaat.com/blog/queue-data-structure/>

<https://www.freecodecamp.org/news/what-is-a-linked-list-types-and-examples/>

<https://stackoverflow.com/questions/840648/why-is-inserting-in-the-middle-of-a-linked-list-o1>

14. Contribución de cada integrante

Antuan: Encargado de toda la lógica (implementación de la pila) y la codificación detrás del problema 1 que corresponde a la pila. Participó en la redacción del README junto a Simón y mejoras notables al informe.

Simon: Encargado de toda la lógica (implementación de la cola) y la codificación detrás del problema 1 que corresponde a la cola. Participó en la redacción del README junto a Antuan y mejoras notables al informe. Adicional, ayudó con las pruebas de el P2.

Sara: Encargada de las pruebas (Tester), organización del repositorio y elaboración del informe.

Se considera un buen trabajo en equipo de forma general :)

15. Link al repositorio de Github

<https://github.com/4KGalaxy/EDA-Practica-1-Estructuras-Unidimensionales>